# SOFTWARE STUDY

## Automatic Color Sorting System

Program logic, algorithms and design choices
of the master ESP32 firmware

Club d'Informatique Industrielle · IGE × EPAC

3 November 2026

# Contents

List of symbols . . . 2
List of tables . . . 3
List of figures . . . 4
Part 1: Introduction . . . 5
1.1 Purpose of the study . . . 5
1.2 Operating principle from the software point of view . . . 5
1.3 General architecture . . . 6
1.4 Organization of the source code . . . 6
Part 2: Study of the basic concepts . . . 6
2.1 Managing time without blocking . . . 7
2.2 State machine and flags . . . 8
2.3 Reading a sensor: threshold, edge and debounce . . . 9
2.4 The circular FIFO queue . . . 9
2.5 Generating signals with hardware PWM . . . 10
2.6 The UART serial link . . . 11
2.7 The I2C bus and the LCD display . . . 12
2.8 Multitasking: two cores, one task, one message queue . . . 12
2.9 Non-volatile memory: `Preferences` . . . 13
2.10 Remote supervision with Blynk . . . 13
2.11 Network time and event log . . . 14
Part 3: Program modules . . . 14
3.1 Module 1: Configuration and parameters . . . 14
3.2 Module 2: Laser sensor acquisition . . . 15
3.3 Module 3: Cube queue and color assignment . . . 16
3.4 Module 4: Reception of the camera orders . . . 18
3.5 Module 5: Belt motor control . . . 19
3.6 Module 6: Bin orientation by the servo . . . 20
3.7 Module 7: Safety . . . 21
3.8 Module 8: Signalling with LEDs and LCD display . . . 23
3.9 Module 9: Supervision by the Blynk cloud . . . 24
3.10 Module 10: Counting, permanent memory and reset . . . 25
Part 4: Integration . . . 26
4.1 Software block diagram . . . 26
4.2 Initialization and main loop . . . 26
4.3 Pin assignment of the microcontroller . . . 27
4.4 Table of Blynk virtual pins . . . 29
4.5 Complete scenario: the passage of a green cube . . . 29
4.6 Test plan . . . 29
4.7 Limits and perspectives . . . 29
Part 5: Conclusion . . . 31
Bibliography . . . 32
Appendix A: Function index . . . 33

# LIST OF SYMBOLS

| | |
|---|---|
| $N_{\text{rev}}$ | : number of steps per motor revolution (200 steps for 1.8°) |
| $\mu$ | : microstepping factor set on the driver (1 in this project) |
| $n$ (`rpm`) | : rotation speed of the drive roller (rev/min) |
| $f_{\text{step}}$ | : frequency of the pulses sent to the PUL input of the driver (Hz) |
| $P$ | : perimeter of the belt drive roller (cm) |
| $v$ | : linear speed of the belt (cm/s) |
| $D$ | : distance between the end sensor and the bin, which is also the minimum spacing between two cubes (cm) |
| $T_{\text{drop}}$ | : drop delay, the time a cube needs to travel $D$ (ms) |
| $S$ | : detection threshold applied to the analog-to-digital converter value |
| $x_{\text{ADC}}$ | : value read on an analog input, from 0 to 4095 (12 bits) |
| $\Delta t_{\text{db}}$ | : debounce duration of a sensor (ms) |
| $t_{\text{now}}$, $t_0$ | : current instant and stored instant, measured by `millis()` (ms) |
| $T_{\text{pause}}$ | : duration of an emergency stop (ms) |
| $T_{\text{PWM}}$ | : period of the servo PWM signal (20 ms) |
| $t_{\text{pw}}$ | : pulse width of the servo signal ($\mu$s) |
| $R$ | : PWM resolution, in bits (14 bits for the servo) |
| $N_{\text{max}}$ | : capacity of the cube queue (10) |
| $f_{\text{msg}}$ | : maximum rate of messages sent to the cloud (messages/s) |
| ADC | : analog-to-digital converter |
| FIFO | : *First In, First Out* queue |
| GPIO | : general-purpose input/output pin |
| HSV | : hue-saturation-value color space, used by the camera |
| I2C | : two-wire serial bus (SDA, SCL), used for the LCD |
| LCD | : liquid crystal display, 16 columns by 2 lines |
| LEDC | : hardware PWM peripheral of the ESP32 |
| NTP | : network time synchronization protocol |
| NVS | : non-volatile storage of the ESP32 (accessed through the `Preferences` library) |
| PUL, DIR, ENA | : pulse, direction and enable inputs of the TB6600 driver |
| PWM | : pulse-width modulation |
| RTOS | : real-time operating system (here FreeRTOS) |
| UART | : asynchronous serial link between the camera and the ESP32 |
| V-pin | : Blynk virtual pin (V0, V1, …), data channel with the dashboard |
| 8N1 | : serial frame with 8 data bits, no parity, 1 stop bit |
| J, V, B, X | : color codes exchanged with the camera, from the French names: *Jaune* (yellow), *Vert* (green), *Bleu* (blue), unknown |
| A | : emergency stop code sent by the camera |
| `?` | : color not yet assigned to a cube in the queue |

# List of tables


1 Walkthrough of a cube and the associated software processing. . . . . 5
2 Organization of the source file and corresponding module. . . . . 7
3 System states and the flags that define them. . . . . 8
4 States of a laser sensor and software interpretation. . . . . 9
5 Protocol of the camera → ESP32 link. . . . . 11
6 The two directions of exchange with Blynk. . . . . 13
7 Tuning parameters of the program. . . . . 14
8 Fields of the `Cube` structure. . . . . 17
9 Effect of the speed on the step frequency, the belt speed and the drop delay. . . . . 20
10 Bin positions: angle, pulse width and duty cycle coded on 14 bits. . . . . 21
11 The four safeties of the system. . . . . 22
12 LED control according to the system state. . . . . 23
13 Messages shown on the LCD display (16 characters per line). . . . . 24
14 Order of the steps of the main loop and justification. . . . . 28
15 Pin assignment of the master ESP32. . . . . 29
16 Virtual pins of the dashboard and processing rules. . . . . 30
17 Walkthrough of a green cube, from detection to counting. . . . . 30
18 Test programs and debugging facilities. . . . . 31
19 Functions of the program, module they belong to and role. . . . . 33

## List of figures


1 Communication diagram: who talks to whom, and by which means. . . . . 6
2 State diagram and transitions. . . . . 8
3 Edge detection with debounce: a bounce at the arrival of the cube is rejected. . . . 10
4 Circular queue of $N_{\max} = 10$ slots holding five cubes: `cubesOnBelt` = 5. Slots 7, 8 and 9 then 0 and 1 are occupied; `?` denotes a color not yet received. . . . . 10
5 Distribution of the work between the two cores and means of communication. . . . 12
6 Algorithm of `risingEdge()`: edge detection with debounce. . . . . 16
7 Life cycle of a cube in the queue. The bin is oriented on arrival if the cube is at the head of the queue, otherwise as soon as the previous cube is sorted. . . . . 17
8 Routing of a received byte. . . . . 18
9 Postponed orientation of the bin for two close cubes: the bin changes only when A has finished falling. . . . . 20
10 Freezing of the timers: the drop delay does not count the time spent in pause. . . . 22
11 Logic of the counter reset. . . . . 26
12 Software block diagram: inputs, central state, outputs. . . . . 27
13 Organization of the main loop. . . . . 28

# PART 1: INTRODUCTION

## 1.1 Purpose of the study

The electrical study fixed the hardware: sensors, driver, motor, servo, display. This document covers the other half of the project, the *program* that brings this hardware to life. It does not simply describe the code line by line: it explains the *logic* used to write it, that is, the problems encountered (waiting without blocking, ignoring bounces, tracking several cubes at once, staying responsive despite an unreliable Wi-Fi, stopping safely) and the solution chosen for each.

The approach follows that of the electrical study. Part 2 presents the basic notions required. Part 3 splits the program into ten modules, each presented with the same outline: role, tools used, diagram, operation and sizing. Part 4 shows how these modules are assembled in the main loop, then gives the reference tables (pins, virtual pins, test plan).

## 1.2 Operating principle from the software point of view

For the program, the system is a sequence of six steps (figure 1 for the links, table 1 for the walkthrough).

TABLE 1. Walkthrough of a cube and the associated software processing.

| No. | Step | Event | Processing |
|---|---|---|---|
| 1 | Detection | Edge on the start laser | A cube is created in the queue, delayed belt start |
| 2 | Identification | Byte received from the camera (J, V, B or X) | Color assigned to the oldest cube without a color |
| 3 | Transport | Time passing | The belt runs at constant speed; each cube keeps its own state |
| 4 | End of belt | Edge on the end laser | The cube is marked as arrived; the bin is oriented if the cube is at the head |
| 5 | Drop | Delay $T_{\text{drop}}$ elapsed | The cube is considered sorted and removed from the queue |
| 6 | Counting | End of the sort | Counters, permanent memory, dashboard |

The program is *event-driven*: it does not run a fixed scenario, it reacts to what happens (an edge on a sensor, a byte on the serial link, a dashboard command, an elapsed delay) by updating a state, then driving the outputs from that state. Three guiding ideas structure all the code:

— **never block**: no `delay()` in the sorting logic; waits are time comparisons, and the network runs in a separate task;

— **one state, several views**: the LEDs, the display and the dashboard are all computed from the same state variables, so they cannot contradict each other;

— **safety first**: in the main loop, the emergency stop is examined before any other action.

### 1.3 General architecture

Figure 1 shows who talks to whom. The ESP32 is the master: it reads the sensors, drives the belt and the bin, displays the status and talks to the cloud. The XIAO ESP32-S3 board is dedicated to vision: from the camera image and the ultrasonic measurement, it determines the color of the cube (by hue, in HSV space) and sends it to the ESP32 over the UART link.

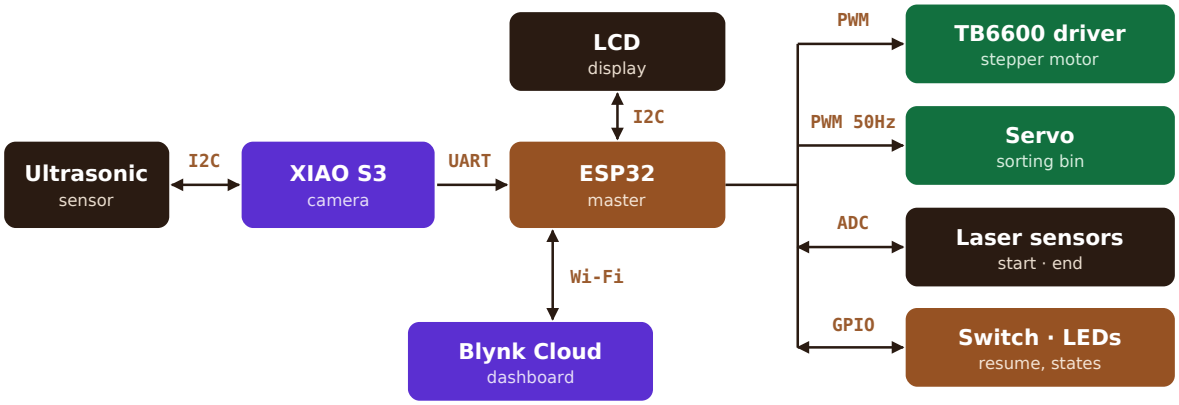



FIGURE 1. Communication diagram: who talks to whom, and by which means.

Each link uses a means of communication suited to its role:

— **UART** (camera → ESP32): one byte per message, at 9600 baud; simple and inexpensive;
— **PWM** (ESP32 → driver and servo): the signal is produced by the hardware peripheral, without occupying the processor;
— **ADC** (laser sensors → ESP32): the analog value is read and compared with a threshold by the program;
— **I2C** (ESP32 → LCD): only two wires for the display;
— **GPIO**: resume switch and status LEDs;
— **Wi-Fi** (ESP32 ↔ Blynk cloud): two-way exchange with the dashboard.

> **Scope.** This study covers the firmware of the master ESP32. The vision part (image acquisition, HSV thresholds, ultrasonic sensor) runs on the XIAO board and is the subject of a separate document; it appears here only through its interface, the UART link described in section 2.6. The electrical stage is described in the electrical system study.

### 1.4 Organization of the source code

The program fits in a single file, split into numbered sections (table 2). This order is not arbitrary: constants come before the structures that use them, and utility functions come before the ones that call them, because the compiler reads the file from top to bottom.

## PART 2: STUDY OF THE BASIC CONCEPTS

This part presents, independently of the project, the embedded-programming notions on which the code relies. Each notion ends with an “In this project” paragraph that indicates where and why it is used.

TABLE 2. Organization of the source file and corresponding module.

| Section | Content | Module (part 3) |
|---|---|---|
| 0 | Blynk identifiers (before the `##include` lines) | 1 Configuration |
| 1, 2 | Pin assignment and tuning parameters | 1 Configuration |
| 3 | System state, `Cube` structure, cube queue | 3 Cube queue |
| 3bis | Blynk task, messages and flags | 9 Blynk supervision |
| 4 | Time management | Part 2 |
| 5 | Motor, servo, LCD, color names | 5, 6 and 8 |
| 5bis | Counters, memory, state sent to the dashboard | 9 and 10 |
| 6 | Laser sensors (edge, debounce) | 2 Sensors |
| 7 | Status and alert LEDs | 8 Signalling |
| 8 | Emergency stop and resume | 7 Safety |
| 8bis | Commands coming from the dashboard | 9 Blynk supervision |
| 9 | `setup()`: initialization | 4.2 |
| 10 | Logic: cubes, colors, end of belt, error | 3, 4 and 7 |
| 11 | `loop()`: main loop | 4.2 |

## 2.1 Managing time without blocking

### 2.1.1 The problem with `delay()`

A program that calls `delay(1000)` does nothing else for one second: it no longer reads its sensors, no longer receives the camera and no longer reacts to the emergency stop. On a moving belt, a cube can pass a sensor during that time without being seen.

### 2.1.2 The solution: compare instants

The `millis()` function returns the number of milliseconds elapsed since startup, as an unsigned 32-bit integer. Instead of waiting, we store the instant $t_0$ at which an event took place, then test on each pass of the loop whether the desired duration has elapsed:

$$\Delta t = (t_{\text{now}} - t_0) \bmod 2^{32}. \tag{1}$$

The program keeps running between two tests; waiting has become a simple comparison.

```
unsigned long elapsedSince(unsigned long t) { return millis() - t; }

if (elapsedSince(lastHeartbeat) < HEARTBEAT_MS) return;   // not yet: carry on
```

The counter returns to zero after $2^{32}$ ms, about 49.7 days. The subtraction of two unsigned 32-bit integers is done modulo $2^{32}$: the result remains correct even if `millis()` has wrapped around in the meantime, provided the measured interval is shorter than 49.7 days. Example: $t_0 = 4\,294\,967\,000$ and $t_{\text{now}} = 200$ after wrap-around give $\Delta t = 200 - 4\,294\,967\,000 + 2^{32} = 496$ ms, which is correct.

> **In this project.** This pattern is used everywhere: delayed belt start, sensor debounce, LED blinking, display refresh, periodic status line, cube timers. The `elapsedSince()`

function re-reads `millis()` on each call, rather than reusing a value read at the beginning of the loop: a write to the display or to flash memory can take long enough to make a stored value stale.

## 2.2 State machine and flags

An automatic system behaves differently depending on its state: it does not react in the same way to a sensor when it is running or in emergency stop. The most readable way to reason about this is the state machine: a set of states, and transitions triggered by events.

The program has no single "state" variable. It combines four boolean flags (table 3), which avoids listing every transition: each flag is modified in a single place in the code.

TABLE 3. System states and the flags that define them.

| State | Condition | Meaning |
|---|---|---|
| PAUSED | `paused` true | Emergency stop: motor cut, everything ignored except resume |
| ERROR | `cubeError` true, `paused` false | An unknown cube blocks the end sensor; waiting for manual removal |
| RUNNING | `beltActive` true, the other two false | The belt runs and cubes are sorted |
| STOPPED | none of the previous cases | Belt stopped, waiting for a cube |

The order of the table is the order of priorities: `paused` overrides `cubeError`, which overrides `beltActive`. The `beltStartPending` flag is not a state but a transitional step: a cube has just been detected and the belt start is scheduled after a short delay. Figure 2 summarizes the transitions.

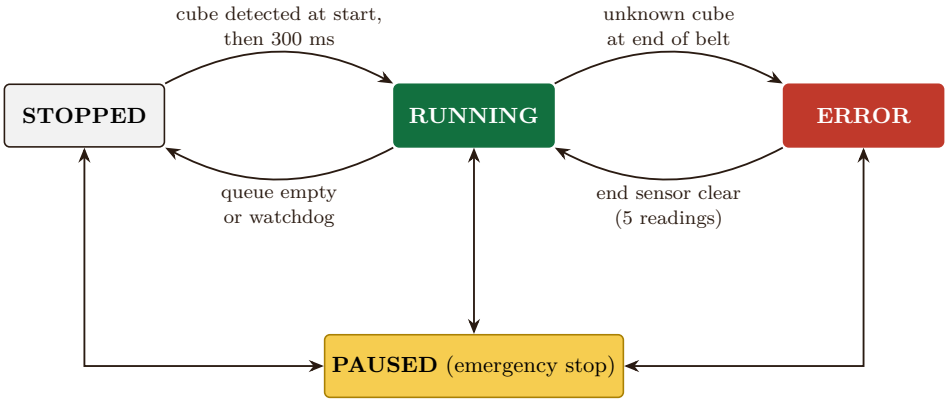



FIGURE 2. State diagram and transitions.

**In this project.** The LEDs, the display and the dashboard are all computed from the same expression, which defines the "belt running" state:

```
bool running = beltActive && !paused && !cubeError;
```

A single computation therefore feeds the three outputs: they cannot contradict each other.

## 2.3 Reading a sensor: threshold, edge and debounce

### 2.3.1 From the analog signal to the "cube present" information

The laser sensors are read by the analog-to-digital converter (ADC) of the ESP32, which returns a value $x_{\mathrm{ADC}}$ from 0 to 4095 (12 bits). The received beam gives a high value, about 4095; a cube that cuts it makes the value drop to about 76. The program chooses a threshold $S = 2000$ and decides:

$$\text{cube present} \iff x_{\mathrm{ADC}} < S \qquad (\text{"active low" sensor}). \tag{2}$$

The threshold lies between the two observed levels, whose midpoint is $(4095 + 76)/2 \approx 2086$. An `activeLow` flag allows the logic to be inverted if the circuit is wired the other way.

TABLE 4. States of a laser sensor and software interpretation.

| Situation | $x_{\mathrm{ADC}}$ | Comparison | Cube present |
|---|---|---|---|
| Beam received (idle) | $\approx 4095$ | $x_{\mathrm{ADC}} > S$ | no |
| Beam cut by a cube | $\approx 76$ | $x_{\mathrm{ADC}} < S$ | yes |

### 2.3.2 Level detection or edge detection

As long as the cube stays in front of the sensor, the "present" level remains true for many passes of the loop. If the program reacted to the level, it would create a new cube on every pass. It must react to the *rising edge*: the precise instant at which the state goes from "absent" to "present". The program therefore stores the previous state and reports only the transition.

### 2.3.3 Debounce

When the beam is cut, the value may oscillate around the threshold for a short time (vibration, edge of the cube, electrical noise). Each oscillation crosses the threshold and would produce a false edge. Debouncing consists in ignoring any new edge for a duration $\Delta t_{\mathrm{db}}$ after the last accepted edge. An edge is therefore accepted if, and only if:

$$\underbrace{x_{\mathrm{ADC}}(k) < S}_{\text{present}} \;\wedge\; \underbrace{\neg\,\mathrm{present}(k-1)}_{\text{absent before}} \;\wedge\; \underbrace{t_{\mathrm{now}} - t_{\mathrm{last}} > \Delta t_{\mathrm{db}}}_{\text{outside the window}}. \tag{3}$$

> **In this project.** Two sensors use this mechanism, with different durations: $\Delta t_{\mathrm{db}} = 600$ ms at the start of the belt and 300 ms at the end of the belt (see the sizing in section 3.2). The sensors are connected to pins 32 and 33, which belong to ADC1: on the ESP32, ADC2 cannot be used while Wi-Fi is active. At startup, the "previous" state is initialized with the real state of the sensors, which avoids detecting a false cube if an object is already in front of the beam.

## 2.4 The circular FIFO queue

Several cubes are on the belt at the same time, each with its own color and its own state. Since the belt is linear, the first cube in is also the first to reach the end of the run: this is the principle of a FIFO queue (first in, first out).

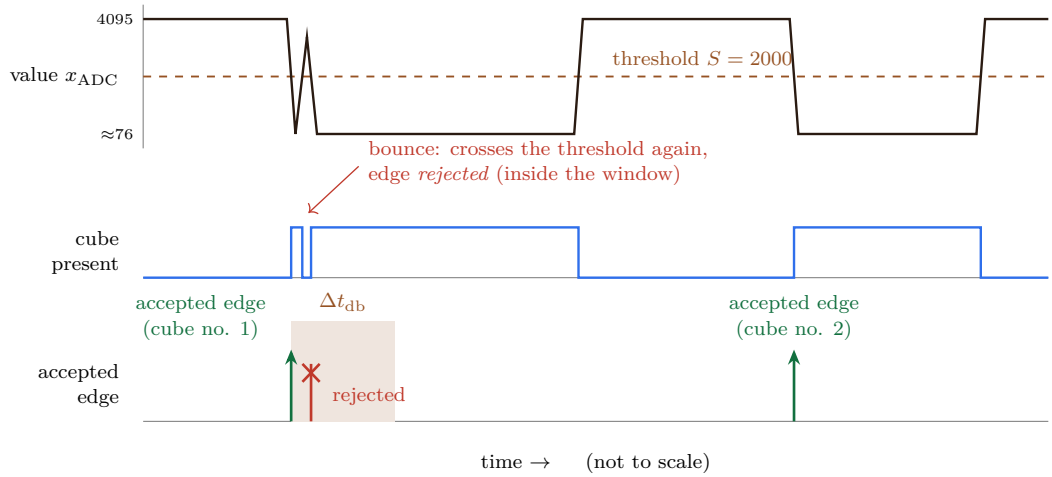



FIGURE 3. Edge detection with debounce: a bounce at the arrival of the cube is rejected.

### 2.4.1 Static array and indices

Rather than dynamic allocation, which fragments memory over time, we use a fixed-size array of $N_{\max}$ elements and three integers: `queueHead` (index of the oldest element), `queueTail` (index of the next free slot) and `cubesOnBelt` (number of elements). When an index reaches the end of the array, it returns to 0: the "modulo" is what makes the queue circular.

$$\text{push:}\quad \mathrm{tab}[\mathrm{tail}] \leftarrow x;\quad \mathrm{tail} \leftarrow (\mathrm{tail}+1) \bmod N_{\max};\quad \mathrm{n} \leftarrow \mathrm{n}+1 \tag{4}$$

$$\text{pop:}\quad \mathrm{head} \leftarrow (\mathrm{head}+1) \bmod N_{\max};\quad \mathrm{n} \leftarrow \mathrm{n}-1 \tag{5}$$

$$\text{element no. } i\text{:}\quad \mathrm{index} = (\mathrm{head}+i) \bmod N_{\max} \tag{6}$$

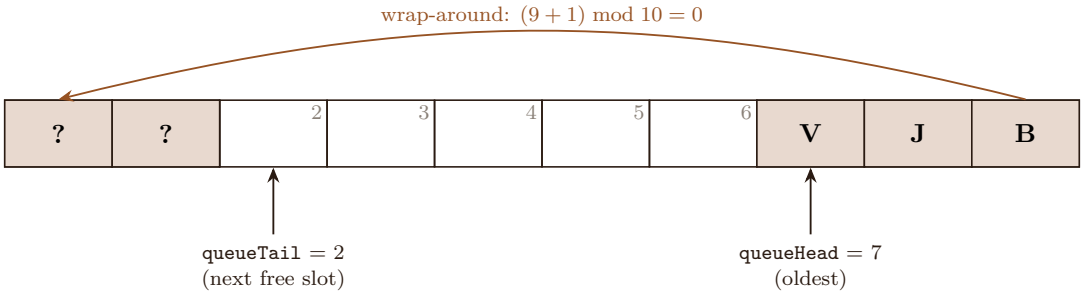



FIGURE 4. Circular queue of $N_{\max} = 10$ slots holding five cubes: `cubesOnBelt` = 5. Slots 7, 8 and 9 then 0 and 1 are occupied; `?` denotes a color not yet received.

> **In this project.** Each slot holds a `Cube` structure of 12 bytes, i.e. 120 bytes for the whole queue. The size of 10 cubes is a compromise: beyond that, a much longer belt than the one of the project would be needed (see section 3.3). Removing a cube in the middle of the queue, which happens after an unknown-cube error, is done by shifting the following ones by one slot.

## 2.5 Generating signals with hardware PWM

PWM (pulse-width modulation) is a square signal characterized by its frequency and its duty cycle. The ESP32 has a dedicated peripheral, the LEDC, that generates these signals without

processor intervention: it is given a frequency, a resolution in bits and a duty value, then works on its own. A duty cycle coded on $R$ bits ranges from 0 to $2^R - 1$.

Two different uses coexist in the project:

— **Pulse train for the motor**: the frequency is the useful quantity (each pulse = one step). The frequency is set with `ledcWriteTone()`, the duty cycle remaining at its default.

— **Servo position**: the frequency is fixed (50 Hz) and it is the pulse *width* that sets the angle. With $R = 14$ bits:

$$\text{duty} = \frac{t_{\text{pw}}}{T_{\text{PWM}}}\,(2^R - 1) = \frac{t_{\text{pw}}}{20\,000\ \mu\text{s}} \times 16\,383. \tag{7}$$

```
ledcAttachChannel(PIN_PUL,   1000, 8, 0);   // motor pulses, channel 0
ledcAttachChannel(PIN_SERVO,   50, 14, 2);  // servo, 14 bits, channel 2

ledcWriteTone(PIN_PUL, (uint32_t)stepFrequencyHz());   // motor: set frequency
ledcWrite(PIN_SERVO, duty);                            // servo: set pulse width
```

**In this project.** The motor test program (`02_stepper_tb6600`) produces each pulse by hand with `micros()`: the processor is permanently busy with the motor. In the main program, the LEDC replaces this method: the processor writes the frequency once, then remains free for the sensors. The channels are chosen by hand (0 for the motor, 2 for the servo), because channels 0 and 1 share the same timer, and the motor and the servo have different frequencies.

## 2.6 The UART serial link

The UART is an asynchronous serial link: the transmitter and the receiver have no common clock but agree on the same speed (in baud). An "8N1" frame sends 8 data bits, no parity bit and 1 stop bit; adding the start bit, one byte occupies 10 bits. At 9600 baud:

$$\text{throughput} = \frac{9600}{10} = 960 \text{ bytes/s}, \qquad \text{duration of a byte} = \frac{10}{9600} \approx 1.04 \text{ ms}. \tag{8}$$

The camera sends a single character per message, which avoids any need for frame splitting or checksum (table 5). On the ESP32 side, reading never blocks: whatever is available is drained, then the program carries on.

Table 5. Protocol of the camera → ESP32 link.

| Byte | Meaning | Effect |
|---|---|---|
| `J` | Yellow | Color assigned to the oldest cube without a color |
| `V` | Green | same |
| `B` | Blue | same |
| `X` | Unknown | same, plus three alert blinks; the cube will stop the belt at the end of its run |
| `A` | Emergency stop | Immediate switch to PAUSED |
| others (carriage return, etc.) | Not applicable | Ignored |

> **In this project.** The code accepts lowercase letters (conversion with `toupper`). The same entry point, the `receiveByte()` function, is used for the UART and for the serial monitor keyboard: the whole sorting process can therefore be tested without a camera by typing J, V, B, X or A. Finally, a lost byte is not dangerous: the cube keeps the color `?`, treated as unknown, which causes a safe stop at the end of the belt rather than a sort into the wrong bin.

## 2.7 The I2C bus and the LCD display

I2C connects a master to several peripherals with only two wires: SDA (data) and SCL (clock). Each peripheral answers to an address; the display's address is `0x27`. The 16×2 display shows two lines of 16 characters; the program buffers are therefore 17 bytes long (16 characters and the end-of-string character), and `snprintf()` cleanly truncates any message that is too long.

> **In this project.** Three protections make the display reliable in an environment where motors generate electrical noise: the bus is set to 100 kHz, a speed more tolerant to noise; a maximum delay of 100 ms (`Wire.setTimeOut(100)`) prevents a stuck bus from freezing the program; and the last message is kept in memory, then rewritten in full every 5 s after re-initializing the display, which erases any stray characters.

## 2.8 Multitasking: two cores, one task, one message queue

### 2.8.1 The problem

The Blynk library must be called regularly (`Blynk.run()`), and this call can take a long time when the Wi-Fi is slow or reconnecting. If it were called in the main loop, an unreliable network would immobilize the belt and the sensors.

### 2.8.2 The solution

The ESP32 has two cores and a real-time system, FreeRTOS, which makes it possible to launch independent *tasks*. The `loop()` function runs on core 1; a task dedicated to Blynk is pinned to core 0. The two communicate only through two mechanisms (figure 5):

— **a message queue** to go from `loop()` to the cloud: `loop()` drops a message, the Blynk task removes it and sends it. This is the producer–consumer scheme. The `xQueueSend(..., 0)` call has a zero wait time: if the queue is full, the message is lost but `loop()` never waits;

— `volatile` **flags** to go from the cloud to `loop()`: the `BLYNK_WRITE` functions (executed in the Blynk task) set variables, which `loop()` reads and resets. The `volatile` keyword forbids the compiler from keeping the variable in a register: without it, `loop()` might never see the change made by the other core.

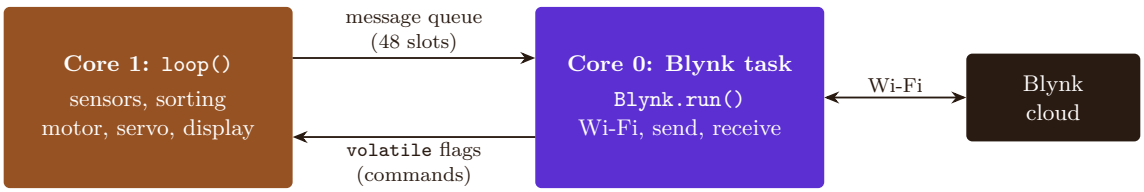



FIGURE 5. Distribution of the work between the two cores and means of communication.

### 2.8.3 Write order

The code of a dashboard message (a text) is copied into a buffer, then the "code received" flag is set *last*. This way `loop()`, which tests the flag first, never reads a half-written buffer. This rule ("data first, flag afterwards") is enough here because there is only one writer and one reader per variable; with several writers, a lock (*mutex*) would be needed.

## 2.9 Non-volatile memory: `Preferences`

RAM is erased at every power loss. So that the sorting counters survive a restart, they are saved in flash memory by the `Preferences` library (NVS system). It works with key–value pairs, grouped in a namespace.

```
prefs.begin("tri", false);          // namespace "tri", read/write
cntJ = prefs.getUInt("J", 0);       // read, 0 if the key does not exist
prefs.putUInt("J", cntJ);           // write
```

Flash supports a limited number of write cycles (of the order of 100 000 erase cycles per sector for memories of this type); the NVS system spreads writes over several sectors to delay wear. Unnecessary writes are nevertheless avoided: in this project, writing happens only at each sorted cube or at each reset, never at every loop pass.

## 2.10 Remote supervision with Blynk

Blynk is a platform that provides a web and mobile dashboard without a server to maintain. It relies on four notions:

— a **template** that describes the product, identified by `BLYNK_TEMPLATE_ID` and `BLYNK_TEMPLATE_NAME`;
— an **authentication token** (`BLYNK_AUTH_TOKEN`) that associates the device with the template; these three definitions must appear before the Blynk `##include` lines;
— **virtual pins** V0, V1, …: data channels, each linked to a dashboard component (*widget*);
— two directions of exchange (table 6).

Table 6. The two directions of exchange with Blynk.

| Direction | Mechanism | Example in the project |
|---|---|---|
| Upstream (ESP32 → cloud) | `Blynk.virtualWrite(pin, value)`, called by the Blynk task after going through the queue | Counters on V1, V5 to V8 |
| Downstream (cloud → ESP32) | `BLYNK_WRITE(Vn)`: function called back on each change of the component | Play/Pause button on V10 |

The service limits the number of messages (about 10 per second). Two rules follow: space out the sends (here at least 110 ms apart) and send the state of the LEDs and of the button only when it *changes*.

### 2.11 Network time and event log

The ESP32 has no calendar clock: it only knows the time elapsed since startup. To timestamp the events of the dashboard log, it queries a time server (NTP) with `configTime()`, with the offset of Benin, UTC+1 without daylight saving (3 600 s). The `getLocalTime(&t, 0)` call has a zero wait time: if the time is not yet known, the program does not wait and displays instead the time elapsed since startup, in the form "+HH:MM:SS". On the serial port, each line starts with the time in milliseconds thanks to the `LOG()` macro.

## PART 3: PROGRAM MODULES

Each module is presented with the same outline: role, tools used (functions and structures of the code), diagram, operation and sizing. The names of functions and variables are those of the source file, in English, so that the reader can find them directly in the code.

### 3.1 Module 1: Configuration and parameters

#### Role

This module gathers at the top of the file everything that can be tuned: pin assignment, speed, geometry, thresholds, durations, network identifiers. The logic never uses a "hard-coded" numeric value: it uses a constant name. Tuning the system therefore only requires modifying this area, without touching the algorithms.

#### Tools used

Constants declared `const`, grouped by theme (motor, geometry, servo, sensors, safety, signalling, debugging, network). The three Blynk identifiers are defined before the `##include` lines, as the library requires.

#### Operation

Constants are evaluated at compile time and cost no RAM. The only quantities modifiable at run time are the speed (`rpm`, changed by `setRpm()`) and the system state. Boolean flags enable or disable debugging functions: `KEYBOARD_SIMULATION`, `DEBUG_RAW_UART`, `WATCHDOG_ENABLED`.

#### Sizing

Table 7 lists the main parameters and the reason for their value; the pin assignment is in table 15.

Table 7. Tuning parameters of the program.

| Constant | Value | Role |
|---|---|---|
| `rpm` | 24 rev/min | Rotation speed of the roller |
| `STEPS_PER_REV_MOTOR` | 200 | Steps per motor revolution |
| `MICROSTEP` | 1 | Microstepping set on the driver |
| `ROLLER_PERIMETER_CM` | 10 cm | Roller perimeter (to be measured) |
| `END_TO_BIN_DISTANCE_CM` | 5 cm | End-of-belt to bin distance, also minimum spacing of cubes |
| `ANGLE_J / ANGLE_B / ANGLE_V` | 0 / 90 / 180° | Servo angle for yellow, blue, green |

| Constant | Value | Role |
|---|---|---|
| `ANGLE_HOME` | 90° | Position at startup and after a stop |
| `DETECTION_THRESHOLD` | 2000 | Detection threshold $S$ (out of 4095) |
| `DEBOUNCE_START_MS` | 600 ms | Debounce of the start sensor |
| `DEBOUNCE_END_MS` | 300 ms | Debounce of the end sensor |
| `START_DELAY_MS` | 300 ms | Delay between detection and belt start |
| `INACTIVITY_TIMEOUT_MS` | 120 000 ms | Watchdog delay |
| `TOGGLE_DEBOUNCE_MS` | 50 ms | Debounce of the resume switch |
| `EMERGENCY_HALF_PERIOD_MS` | 250 ms | Half-period of blinking during the emergency stop |
| `ALERT_HALF_PERIOD_MS` | 200 ms | Half-period of the alert blinking |
| `ALERT_BLINK_COUNT` | 3 | Blinks when an `X` is received |
| `CLEAR_SAMPLES_REQUIRED` | 5 | Consecutive "clear" readings to lift an error |
| `LCD_REFRESH_MS` | 5000 ms | Full rewrite of the display |
| `HEARTBEAT_MS` | 5000 ms | Period of the status line on the serial port |
| `BLYNK_INTERVAL_MS` | 110 ms | Minimum interval between two sends to the cloud |
| `MAX_CUBES_ON_BELT` | 10 | Capacity of the queue |
| `RESET_CODE` | "EPAC" | Code to reset the counters |

## 3.2 Module 2: Laser sensor acquisition

### Role

Turn the analog value of each sensor into a reliable event: "a cube has just arrived in front of this beam". This module decides nothing: it supplies events to the other modules.

### Tools used

The `Sensor` structure (pin, debounce duration, last accepted instant, previous state, last value, polarity), the two instances `sensorStart` and `sensorEnd`, and three functions: `cubePresent()`, `risingEdge()` and `initSensors()`. The physical stage (laser, photoresistor) is described in the electrical study, subsystems 1 and 2.

### Diagram

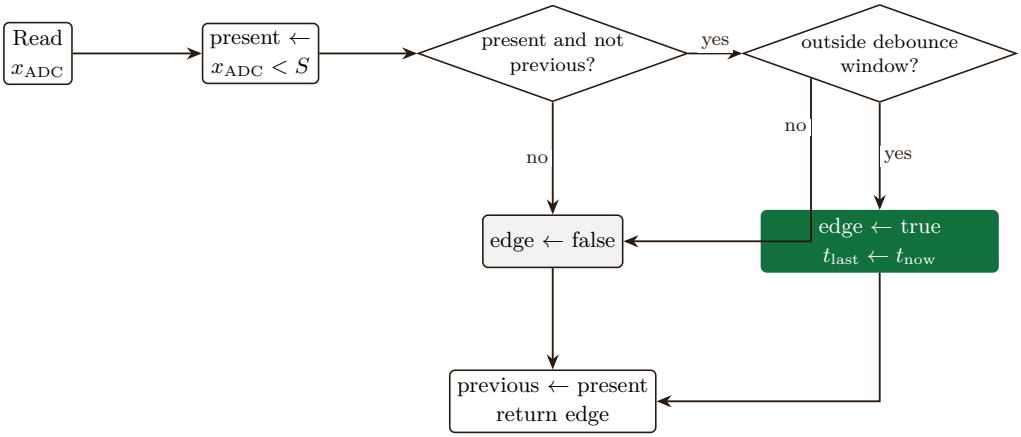



FIGURE 6. Algorithm of `risingEdge()`: edge detection with debounce.

### Operation

On each pass of the loop, `risingEdge()` re-reads the input, determines whether a cube is present, then applies the three conditions of the edge equation: present, absent on the previous pass, outside the debounce window. The "previous" state is updated on *every* call, whether there is an edge or not; without this, a cube remaining in front of the sensor would be seen as a new edge at the first opportunity.

```
bool risingEdge(Sensor &s) {
  s.value = analogRead(s.pin);
  bool present = cubePresent(s);
  bool edge = present && !s.previous && elapsedSince(s.lastMs) > s.debounceMs;
  if (edge) s.lastMs = millis();
  s.previous = present;
  return edge;
}
```

The program also calls `risingEdge()` during the emergency stop, ignoring its result: the "previous" states thus stay up to date, and a cube that remains in front of a sensor during the pause does not produce a false edge at resume.

### Sizing

*Threshold.* $S = 2000$ lies between the high level ($\approx 4095$) and the low level ($\approx 76$), close to their midpoint ($\approx 2086$), which leaves a margin of more than 1900 units on each side to absorb noise.

*Debounce.* It must remain shorter than the minimum time separating two cubes in front of the same sensor, otherwise a real cube would be rejected. With the speed $v = 4$ cm/s (see module 5) and the minimum spacing $D = 5$ cm:

$$T_{\min} = \frac{D}{v} = \frac{5}{4} = 1.25\ \text{s} = 1250\ \text{ms} \quad \Rightarrow \quad \Delta t_{\text{db}} = 600\ \text{ms} < 1250\ \text{ms} \ \text{ and } \ 300\ \text{ms} < 1250\ \text{ms}. \tag{9}$$

The constraint is met with a margin of more than a factor of two. It tightens if the speed increases: at 48 rev/min, $T_{\min} = 625$ ms, and the 600 ms of the start sensor become almost insufficient.

## 3.3 Module 3: Cube queue and color assignment

### Role

Track each cube present on the belt, from its detection at the start until it is sorted, associating with it its color and its progress state. It is the memory of the system: all the other modules consult it.

### Tools used

The `Cube` structure and the circular queue `cubeQueue` (see part 2), with the functions `addNewCube()`, `receiveColor()`, `handleEndOfBelt()`, `advanceToNextCube()`, `removeCubeAt()` and `resetQueue()`.

TABLE 8. Fields of the `Cube` structure.

| Field | Type | Meaning |
|---|---|---|
| `color` | `char` | `J`, `V`, `B`, `X` or `?` (not yet received) |
| `reachedEnd` | `bool` | The end sensor has seen this cube |
| `binAligned` | `bool` | The bin has been oriented for this cube |
| `entryTime` | `unsigned long` | Instant of detection at the start (used for the total transit time) |
| `dropTime` | `unsigned long` | Instant of arrival at the end of the belt, starting point of the drop delay |

### Diagram

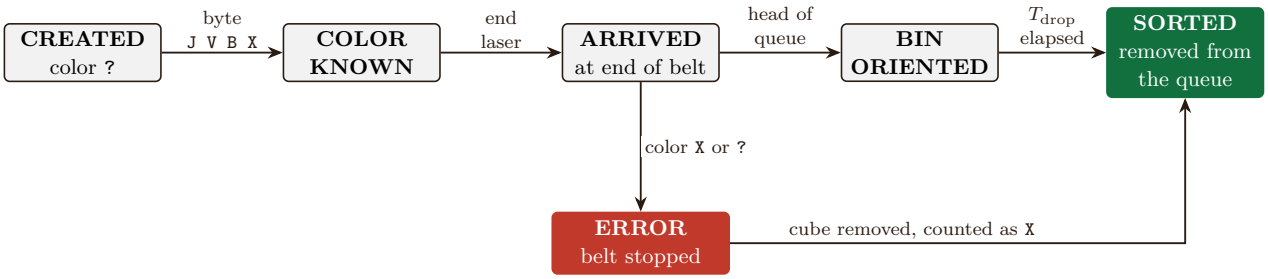



FIGURE 7. Life cycle of a cube in the queue. The bin is oriented on arrival if the cube is at the head of the queue, otherwise as soon as the previous cube is sorted.

### Operation

*Creation.* On each edge of the start sensor, `addNewCube()` appends a cube of color `?` at the tail, whose `entryTime` is the current instant. If the queue is full, the cube is ignored and reported in the log; if the belt is stopped, the delayed start is scheduled.

*Color assignment.* When a color byte arrives, `receiveColor()` scans the queue from the head and gives the color to the *oldest cube still at* `?`. This is the same FIFO reasoning as for the order of arrival: the camera identifies the cubes in the order in which they entered. If no cube is waiting for a color, the byte is ignored.

*Arrival at the end of the belt.* When the end laser detects a cube, `handleEndOfBelt()` looks for the first cube in the queue whose `reachedEnd` is false: it is the one that is arriving, since cubes arrive in order. Three cases arise:

1. unknown color (`X` or `?`): immediate stop of the belt (module 7);
2. cube at the head (position 0): the bin is oriented right away;

3. cube at position $i > 0$: the orientation is *postponed*, because the previous cube is still falling and changing the bin would direct it into the wrong container. It will be done by `advanceToNextCube()` as soon as the head cube is sorted.

```
for (int i = 0; i < cubesOnBelt; i++) {
  int idx = (queueHead + i) % MAX_CUBES_ON_BELT;
  if (!cubeQueue[idx].reachedEnd) {      // first cube not yet seen at the end
    cubeQueue[idx].reachedEnd = true;
    cubeQueue[idx].dropTime = millis();
    /* unknown colour -> error; head of queue -> orient bin; else postpone */
    return;
  }
}
```

### Sizing

*Memory.* A `Cube` structure occupies 12 bytes (one character, two booleans, one padding byte and two 4-byte integers): the complete queue of 10 cubes occupies 120 bytes.

*Capacity.* With a minimum spacing $D = 5$ cm between cubes, a queue of $N_{\max} = 10$ cubes is enough for a belt of about $N_{\max} \times D = 50$ cm. For a longer belt, `MAX_CUBES_ON_BELT` must be increased; the memory required remains negligible.

## 3.4 Module 4: Reception of the camera orders

### Role

Read on the UART link the bytes sent by the XIAO board, and turn them into actions: assign a color or trigger the emergency stop.

### Tools used

The serial port `Serial1` (RX on pin 16, 9600 baud, 8N1) and the functions `handleCameraUart()`, `readCameraByte()`, `receiveByte()` and `receiveColor()`. The `Serial` port (USB) is used for the simulation keyboard.

### Diagram

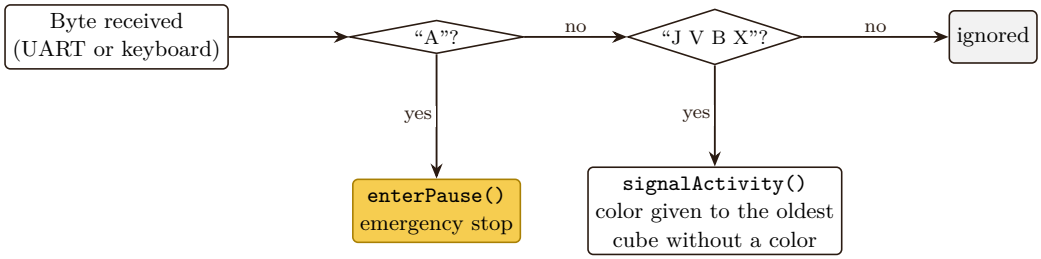



FIGURE 8. Routing of a received byte.

### Operation

`handleCameraUart()` reads all the available bytes (`while` loop) then returns: it never blocks. Each byte goes through `receiveByte()`, which first tests the stop code `A`, which has priority, then forwards to `receiveColor()`. The latter converts the byte to uppercase, discards everything that is not `J`, `V`, `B` or `X`, and reports an activity to the watchdog. An `X` also triggers three alert blinks.

During the emergency stop, received bytes are read then discarded. This is deliberate: a color byte arriving during the pause would be assigned to a cube that may no longer be in the same place at resume, which would shift all the following color–cube associations.

#### Sizing

At 9600 baud, a byte lasts about 1.04 ms. At most one byte therefore arrives per millisecond, and sending one color per cube, at the rate of one cube every 1.25 s at the minimum, is far below the capacity of the link. The reception buffer of the driver (256 bytes by default) cannot overflow as long as the main loop runs faster than the camera transmits.

### 3.5 Module 5: Belt motor control

#### Role

Make the belt advance at constant speed, only when there are cubes to sort, and compute the drop time that follows from this speed.

#### Tools used

The PUL pulse signal produced by the LEDC (pin 5), the ENA enable pin (pin 2) and the functions `startMotor()`, `stopMotor()`, `setRpm()`, `stepFrequencyHz()` and `computeDropDelayMs()`. The direction of rotation is not controlled: the DIR input of the driver is wired directly to the supply, the belt moving in one direction only.

#### Operation

At startup, the ENA pin is set to the level that *enables* the driver (`ENA_ACTIVE_LEVEL`, depending on the wiring) and stays there; only the PUL signal commands the movement. `startMotor()` sets the pulse frequency with `ledcWriteTone()`, and `stopMotor()` sets it to zero.

The start is *delayed*: on detection of the first cube, the `beltStartPending` flag is raised and the instant is stored; the main loop starts the motor when `START_DELAY_MS` (300 ms) has elapsed. The delay is therefore obtained without `delay()`, and the loop keeps listening during this time. The motor stops normally when the queue is empty (`advanceToNextCube()`), and in an emergency in the cases of module 7.

#### Sizing

Three quantities follow from the roller speed $n$:

$$f_{\text{step}} = \frac{n \cdot N_{\text{rev}} \cdot \mu}{60}, \tag{10}$$

$$v = \frac{n}{60} P, \tag{11}$$

$$T_{\text{drop}} = \frac{D}{v} \times 1000 = \frac{60\,D}{n\,P} \times 1000 \text{ ms}. \tag{12}$$

With the values of the project ($n = 24$ rev/min, $N_{\text{rev}} = 200$, $\mu = 1$, $P = 10$ cm, $D = 5$ cm):

$$f_{\text{step}} = \frac{24 \times 200 \times 1}{60} = 80 \text{ Hz}, \qquad v = \frac{24}{60} \times 10 = 4 \text{ cm/s}, \qquad T_{\text{drop}} = \frac{5}{4} \times 1000 = 1250 \text{ ms}. \tag{13}$$

The drop delay is inversely proportional to the speed: doubling the speed halves the time the bin has to orient itself. The computation is therefore redone at each use from the current value of `rpm`, rather than fixed once and for all. If `rpm` is zero or negative, the function returns

TABLE 9. Effect of the speed on the step frequency, the belt speed and the drop delay.

| $n$ (rev/min) | $f_{\text{step}}$ (Hz) | $v$ (cm/s) | $T_{\text{drop}}$ (ms) |
|---|---|---|---|
| 12 | 40 | 2 | 2500 |
| 24 | 80 | 4 | 1250 |
| 48 | 160 | 8 | 625 |

a fallback delay of 2000 ms to avoid a division by zero. The result is truncated to a whole millisecond, which is negligible compared with the 1250 ms.

## 3.6 Module 6: Bin orientation by the servo

### Role

Place the bin under the exit of the belt, in the position that matches the color of the cube about to fall, and never move it while a cube is falling.

### Tools used

The 50 Hz PWM signal on pin 23 and the functions `servoAngle()`, `angleFor()`, `orientBin()`, `menuIndex()` and `colorName()`. The `BIN_MENU` array gives the order of the choices in the "Bin" dropdown list of the dashboard.

### Diagram

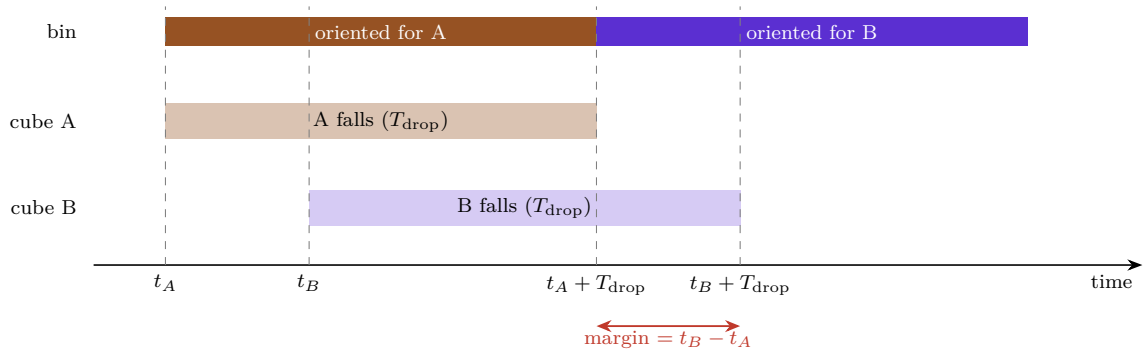



FIGURE 9. Postponed orientation of the bin for two close cubes: the bin changes only when A has finished falling.

### Operation

`servoAngle()` limits the angle between 0 and 180°, computes the pulse width

$$t_{\text{pw}} = 500 + \frac{\text{angle}}{180} \times 1900 \quad (\mu\text{s}), \tag{14}$$

then converts it to a duty cycle with equation (7). `orientBin()` translates a color into an angle (`angleFor()`, which returns $-1$ for a non-sortable color and then orients nothing), positions the servo, stores the bin color, updates the dashboard dropdown list and displays the color on the screen.

Figure 9 shows the essential rule: the bin follows the order of the cubes. When two cubes arrive at the end of the belt at $t_A$ then $t_B$ with $t_B - t_A < T_{\text{drop}}$, A is still falling when B arrives.

The bin therefore stays at the position of A until the instant $t_A + T_{\text{drop}}$; it is then oriented for B, which will fall at $t_B + T_{\text{drop}}$. The margin the servo has to move is $t_B - t_A$.

#### Sizing

*Pulse widths.* The code uses a range of 500 to 2400 $\mu$s, wider than the nominal range of 1 to 2 ms given for the SG90 in the electrical study: it makes it possible to cover the full 180° travel. For the three positions of the project:

TABLE 10. Bin positions: angle, pulse width and duty cycle coded on 14 bits.

| Color | Angle | $t_{\text{pw}}$ ($\mu$s) | Computation | duty |
|---|---|---|---|---|
| Yellow (`J`) | 0° | 500 | $500/20000 \times 16383 = 409.6$ | 409 |
| Blue (`B`) | 90° | 1450 | $1450/20000 \times 16383 = 1187.8$ | 1187 |
| Green (`V`) | 180° | 2400 | $2400/20000 \times 16383 = 1966.0$ | 1965 |

The conversion to an integer truncates: the last line gives 1965 and not 1966, a difference of less than 1.3 $\mu$s.

*Resolution.* One duty step is worth $20\,000/16\,383 \approx 1.22$ $\mu$s, i.e. $1.22 \times 180/1900 \approx 0.12°$: far finer than the mechanical accuracy of the servo.

*Response time.* The margin available to move the servo is at least $D/v = 1250$ ms when cubes respect the minimum spacing. It is far greater than the typical travel time of an SG90 (of the order of 0.1 s for 60°). The program does not wait for the end of the movement: it simply compares the current instant with `dropTime` and reports in the log the remaining margin, or a "cubes too close" warning.

### 3.7 Module 7: Safety

#### Role

Guarantee that in all circumstances the system stops without losing track of the cubes: emergency stop, controlled resume, unknown cube, prolonged inactivity.

#### Tools used

The flags `paused` and `cubeError`, the instants `pauseStart` and `errorStart`, the `PIN_PAUSE` pin (switch), and the functions `enterPause()`, `exitPause()`, `handleToggleSwitch()`, `triggerCubeError()`, `handleCubeError()` and `signalActivity()`.

#### Operation

*Emergency stop.* `enterPause()` raises `paused`, notes the instant, cuts the motor and displays "EMERGENCY STOP". In the main loop, this flag is tested before any other action: as long as it is raised, only the switch, the dashboard commands and the LEDs are handled; received bytes are discarded and the sensors are read idly.

*Freezing the timers.* A cube has a drop delay in progress when the stop occurs; this delay must not keep running during the pause, otherwise the cube would be declared sorted when it has not moved. At resume, `exitPause()` computes the duration $T_{\text{pause}} = t_{\text{resume}} - t_{\text{pause}}$ and *carries it over* to all the stored instants:

$$t'_{\text{stored}} = t_{\text{stored}} + T_{\text{pause}} \quad \Rightarrow \quad t_{\text{now}} - t'_{\text{stored}} \text{ unchanged.} \tag{15}$$

The following are shifted: `entryTime` and `dropTime` of each cube, the instant of the delayed

TABLE 11. The four safeties of the system.

| Safety | Trigger | Action | Way out of the state |
|---|---|---|---|
| Emergency stop | `A` byte (camera button), dashboard Pause button, `A` key on the keyboard | Motor cut, everything is ignored, red LED blinking | Change of the switch position, or Play |
| Unknown cube | Cube of color `X` or `?` detected at the end of the belt | Motor cut on the spot, red LED blinking | End sensor clear (manual removal), automatic resume |
| Watchdog | 120 s without any sign of activity while the belt runs | Stop, bin to center, queue cleared | New cube at the start of the belt |
| Debounce | False edges (sensors, switch) | Ignored | Not applicable |

start and the instant of the start of the error. The time that has elapsed "for the system" is thus identical before and after the pause (figure 10).

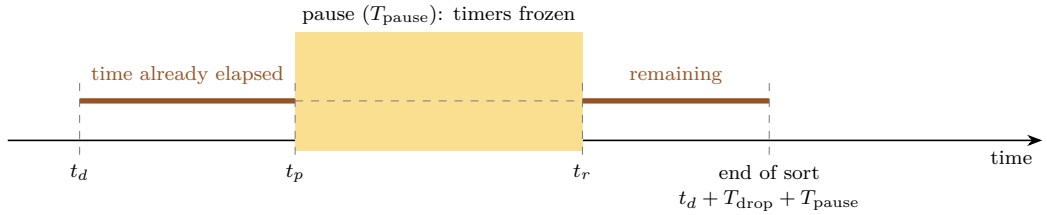



FIGURE 10. Freezing of the timers: the drop delay does not count the time spent in pause.

*Resume with the switch.* Resuming does not require a precise position: it reacts to *any change of level* of the switch, after debounce. The operator therefore does not need to know which position the switch is in. Debouncing stores the "raw" level and the "stable" level: the stable level changes only if the raw level differs and has remained unchanged for at least 50 ms. At startup, the initial level is read again, so that the starting position is not mistaken for a command.

*Unknown cube.* When a cube of color `X` or `?` reaches the end laser, `triggerCubeError()` cuts the motor immediately: the cube is then still in front of the sensor, which allows the operator to remove it. The `handleCubeError()` function waits until the end sensor has been clear for `CLEAR_SAMPLES_REQUIRED` (5) consecutive readings. It then counts the cube in the unknown counter, removes it from the queue, shifts the timers of the other cubes by the duration of the error, prepares the next cube and restarts the motor if cubes remain.

*Watchdog.* At each sign of life (edge on a sensor, color received, cube sorted, resume, belt start, end of error), `signalActivity()` stores the instant and the source. If the belt is active and 120 s pass without any sign of life, the system assumes an abnormal situation (cube stuck outside the sensors, silent camera): it stops the motor, puts the bin back in the central position, empties the queue, displays "EMERGENCY" and logs the cause.

### Sizing

*Watchdog.* Its duration must be longer than the longest interval between two signs of life in normal operation. In this project, the longest interval is the transit of a single cube, from the start sensor to the end sensor; at a speed of 4 cm/s, a belt of length $L$ gives $L/4$ s. At 4 cm/s, 120 s correspond to 480 cm of belt travelled: the margin is very large. It should only be reduced after measuring the real transit time.

*Multiple reading.* Five consecutive readings prevent a single spurious "clear" reading, for example while the cube is being removed at an angle, from restarting the belt too early.

*Switch.* 50 ms of debounce is much longer than usual mechanical bounces (a few milliseconds) and remains imperceptible to an operator.

## 3.8 Module 8: Signalling with LEDs and LCD display

### Role

Show the state of the system at all times to the operator, without needing the dashboard.

### Tools used

Three LEDs (green, yellow, red) on pins 27, 26 and 14, the 16×2 LCD display on I2C, and the functions `updateLeds()`, `updateAlert()`, `triggerAlertX()`, `showLcd()`, `writeLcd()` and `refreshLcd()`.

### Operation

The LEDs are recomputed on each loop pass from the state (table 12). The alert LED is a parameter (`PIN_LED_ALERT`): it can be the red or the yellow one.

Table 12. LED control according to the system state.

| State | Green | Yellow | Red | Blynk V2/V3/V4 |
|---|---|---|---|---|
| RUNNING | steady | off | off | 255 / 0 / 0 |
| STOPPED (waiting) | off | steady | off | 0 / 255 / 0 |
| PAUSED | off | steady | blinking (period 500 ms) | 0 / 255 / 255 |
| ERROR | off | steady | blinking (period 400 ms) | 0 / 255 / 255 |
| Reception of an `X` (running) | steady | off | 3 blinks (1.2 s) | 255 / 0 / 0 |

Blinking is a small non-blocking state machine: a counter of remaining half-cycles and an instant of last change. On each pass, if a half-period has elapsed, the state of the LED is inverted and the counter decreases. For three blinks, the counter is initialized to $2 \times 3 = 6$ half-cycles of 200 ms: the sequence lasts $6 \times 200 = 1200$ ms. During an error, the counter does not decrease, and the blinking continues as long as the error lasts.

TABLE 13. Messages shown on the LCD display (16 characters per line).

| Event | Line 1 | Line 2 |
|---|---|---|
| Startup | `System ready` | `Waiting...` |
| New cube | `New cube` | `On belt: n` |
| Bin oriented | `BIN:` | color |
| Bin chosen from the dashboard | `BIN (manual):` | color |
| Cube sorted | `ITEM SORTED` | color |
| Emergency stop | `EMERGENCY STOP` | `Flip switch` |
| Resume | `RESUMED` | (empty) |
| Unknown cube | `CUBE ERROR` | `Remove the cube` |
| Cube removed | `Cube removed` | `Resuming` |
| Watchdog | `EMERGENCY` | `Idle 120s` |

#### Sizing

*Message length.* All the texts of table 13 fit in 16 characters or fewer (the longest, `Remove the cube`, has 15); copying with `snprintf()` into 17-byte buffers protects against any overflow.

*Blinking frequencies.* During the emergency stop, the half-period of 250 ms gives a period of 500 ms (2 Hz); for the alert, the half-period of 200 ms gives 400 ms (2.5 Hz). The two rhythms are different so that the operator visually distinguishes an emergency from an alert.

*LED current.* Each LED is powered by an ESP32 pin through a series resistor computed in the electrical study (subsystem 8).

### 3.9 Module 9: Supervision by the Blynk cloud

#### Role

Send the system state to the dashboard (counters, LEDs, last sort, log, chart) and receive its commands (Play/Pause, bin choice, reset), without ever slowing down the sorting.

#### Tools used

The `blynkTask()` task on core 0, the `blynkQueue` queue of `Msg` structures, the sending functions `pushInt()`, `pushTxt()` and `termLog()`, the receiving functions `BLYNK_WRITE(V10, V12, V13, V15, V16)` and `BLYNK_CONNECTED()`, and, on the `loop()` side, `handleBlynkCommands()`, `sendFullState()` and `syncBlynkState()`.

### Operation

*Blynk task.* It loops indefinitely. If Wi-Fi is down, it marks the system offline and waits 500 ms. Otherwise, it calls `Blynk.run()`, then, at most every 110 ms, removes a message from the queue and sends it (`virtualWrite`). If Blynk is offline, the queue is emptied: outdated messages are useless, the full state will be resent at reconnection. Each pass ends with a 10 ms pause, which leaves time for the Wi-Fi stack.

*Sending.* `pushInt()` and `pushTxt()` build a message and drop it in the queue without waiting (zero delay). `termLog()` formats a timestamped line "[HH:MM:SS] message" for the log (V12). Sorting therefore only pays the cost of a copy in memory.

*Sending on change.* `syncBlynkState()` keeps in memory the last values of the virtual LEDs and of the Play/Pause button, and sends a message only when a value changes. This saves the throughput allowed by the cloud. A `forceState` flag resets this memory to force a resend.

*Reception.* The `BLYNK_WRITE` functions (executed in the Blynk task) only set a flag or copy a value; the real processing is done by `handleBlynkCommands()`, in `loop()`, at the beginning of each pass. This split guarantees that actions on the motor and the servo happen in a single thread of execution. The rules of each command are in table 16.

*Resynchronization.* At each (re)connection, `BLYNK_CONNECTED()` raises a flag; `loop()` then calls `sendFullState()`, which resends the last sort, the counters, the bin state, resets the reset button and the code field, then forces the LEDs to be resent. The dashboard thus reflects the real state of the system after any interruption.

### Sizing

*Throughput.* The interval of 110 ms limits sending to

$$f_{\text{msg}} = \frac{1}{0.110} \approx 9.1 \text{ messages/s}, \tag{16}$$

below the limit of about 10 messages per second of the service.

*Message queue.* A message (`Msg`) occupies 108 bytes: pin, type, padding, 4-byte integer and 100-character text. The queue of 48 messages therefore occupies $48 \times 108 = 5184$ bytes, about 5 KB. At reconnection, `sendFullState()` drops 10 messages (last sort, five counters, reset button, code field, bin list, one log line) and `syncBlynkState()` adds 4: 14 messages in total, drained in $14 \times 0.110 \approx 1.5$ s. The queue of 48 slots absorbs this burst with a large margin.

*Text size.* The log line is built in a buffer of 80 characters, preceded by a timestamp of 10 characters and a line break; it fits in the 100 characters of a message.

*Task stack.* 8192 bytes are reserved for the Blynk task, a usual value for the network stack.

## 3.10 Module 10: Counting, permanent memory and reset

### Role

Count the sorted cubes by color, keep these counts after a power loss, and allow them to be reset safely.

### Tools used

The counters `cntJ`, `cntV`, `cntB`, `cntX`, the `Preferences` library (namespace `tri`), and the functions `recordSort()`, `saveCounters()`, `pushCounters()`, `totalSorted()` and `resetCounters()`.

### Operation

*Counting.* `recordSort()` is called once per cube: at normal sorting (step 6 of the loop) and when leaving an unknown-cube error, which increments the `X` counter. It increments the counter of the color, saves the four counters in permanent memory, stores the name of the last sort, then sends to the dashboard the last sort, the counters, the total (chart V11) and a log line. At startup, the counters are read back from memory: a restart therefore loses nothing.

*Reset with double validation.* Erasing the production count is a sensitive action. It requires two distinct gestures on the dashboard (figure 11): arming the "Reset DB" switch (V13), then entering the code in the "Confirmation Code" field (V16). Entering the code without the switch armed is ignored; a wrong code cancels the arming. In every case, the switch is set back to OFF and the input field is cleared. The comparison ignores case (`strcasecmp`) and the spaces around the text.

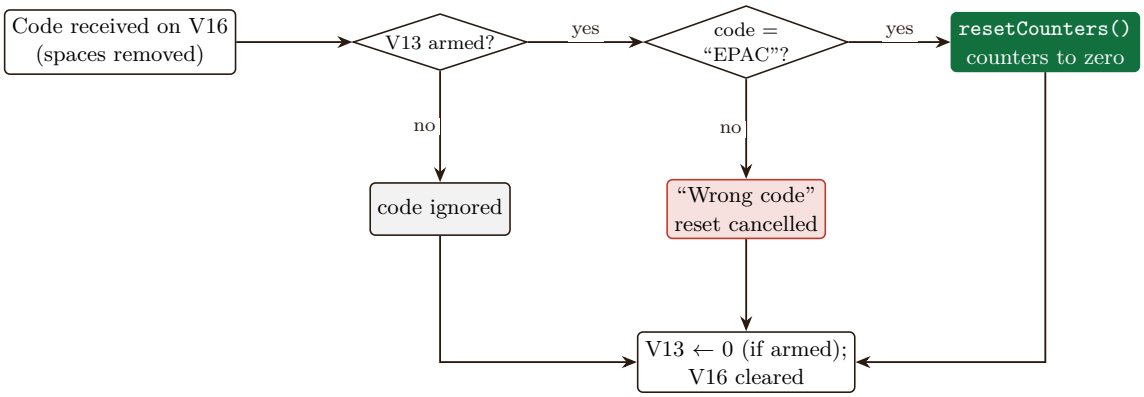



FIGURE 11. Logic of the counter reset.

### Sizing

*Counter capacity.* They are unsigned 32-bit integers, up to $2^{32} - 1 = 4\,294\,967\,295$: overflow is out of reach.

*Flash writes.* Four values are written for each sorted cube. At the maximum rate of one cube every 1.25 s (0.8 cube per second), this means at most 3.2 value writes per second at saturation; flash wear should be checked if the system runs continuously for a long time (see part 4, limits and perspectives).

# PART 4: INTEGRATION

## 4.1 Software block diagram

Figure 12 shows how the ten modules fit together. The inputs (sensors, camera, switch, dashboard) feed the central state of the system (the cube queue, the flags and the timers); the output modules draw their commands from it. The safeties have control over the motor at all times.

## 4.2 Initialization and main loop

### 4.2.1 Initialization: `setup()`

It runs once, in a precise order. First the USB port (115200 baud) and the camera port, then the power outputs: the driver is enabled, the motor PWM is configured then set to stop, the

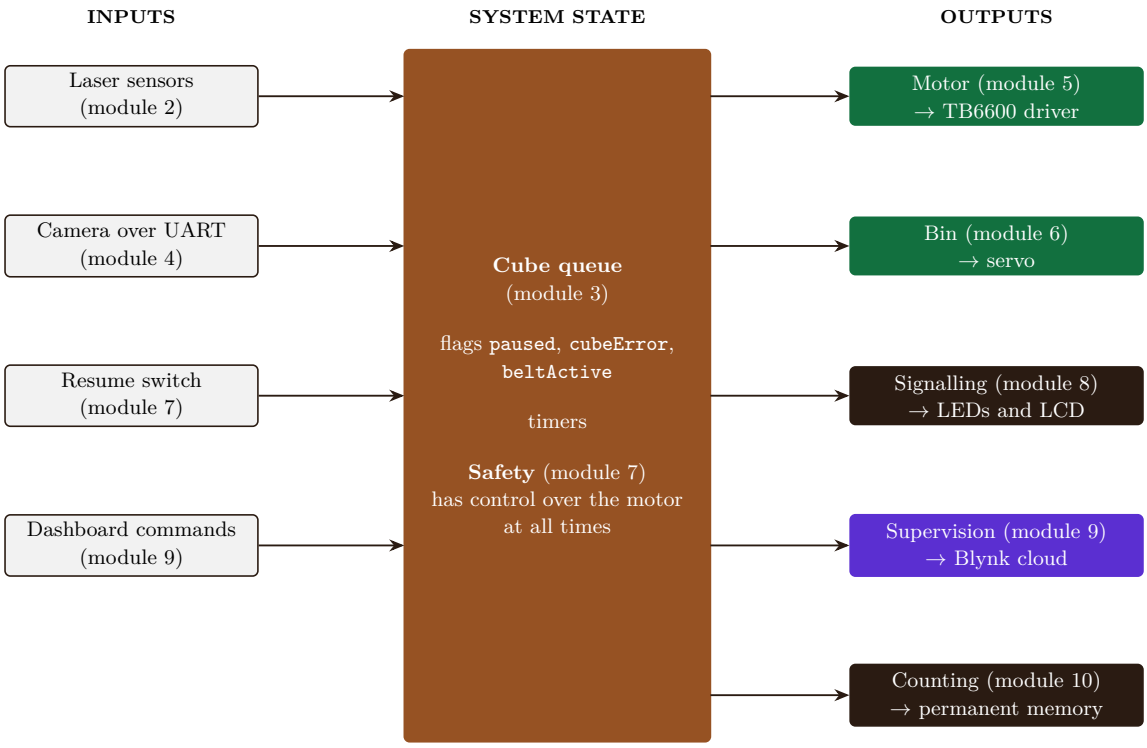



FIGURE 12. Software block diagram: inputs, central state, outputs.

servo is placed in the central position. The inputs (sensors, switch), the LEDs and the I2C bus follow, then the display shows "System ready". The real state of the sensors and of the switch is read *before* the logic starts, to avoid false detections. The counters are read back from memory. Finally, Wi-Fi is started without waiting, the network time is requested and the Blynk task starts. The power outputs are therefore in a safe state (motor stopped) before any processing.

### 4.2.2 Main loop: `loop()`

The loop repeats indefinitely the sequence of table 14. Its order is not arbitrary: each step assumes that the previous ones have already taken place.

### 4.2.3 Two points of logic to remember

*Detect before assigning.* The sensors are read (step 1) before the camera (step 2): if a cube has just entered and its color arrives in the same pass, the record already exists and the color is correctly assigned.

*Test the pause twice.* The pause is tested at the top of the loop, then again after reading the camera: an `A` byte read at step 2 must stop the current pass, without letting the delayed start or the processing of the end sensor run once more.

## 4.3 Pin assignment of the microcontroller

> **Two points of caution.** GPIO2 is a boot-configuration pin of the ESP32 (*strapping pin*): the circuit connected to ENA must not force an incorrect level during startup. The analog inputs use ADC1 because ADC2 cannot be used when Wi-Fi is active.

TABLE 14. Order of the steps of the main loop and justification.

| No. | Step | Why at this place |
|---|---|---|
| 0 | Switch, dashboard commands, status line, display | Must run even in pause, otherwise it would be impossible to resume |
| P | If PAUSED: discard bytes, read sensors idly, LEDs, *return* | The pause neutralizes the rest of the loop |
| 1 | Reading of the two sensors | A detected cube creates its record *before* its color arrives |
| 2 | Camera and keyboard reception | The color finds a cube without a color to be assigned to; an `A` can raise the pause |
| 2b | If PAUSED (new): LEDs, *return* | An `A` has just arrived: everything stops immediately |
| 2c | If ERROR: wait for removal, LEDs, *return* | The belt is cut, nothing else must advance |
| 3 | Watchdog | Measures inactivity after taking into account the signs of life of steps 1 and 2 |
| 4 | Delayed belt start | The 300 ms delay is tested on each pass |
| 5 | Processing of the end sensor | The color is known (step 2), the belt is running (step 4) |
| 6 | End of the head cube's drop delay | Declares the cube sorted, counts, prepares the next cube |
| 7 | LEDs and state sent to the dashboard | Reflects the state once all the decisions have been made |

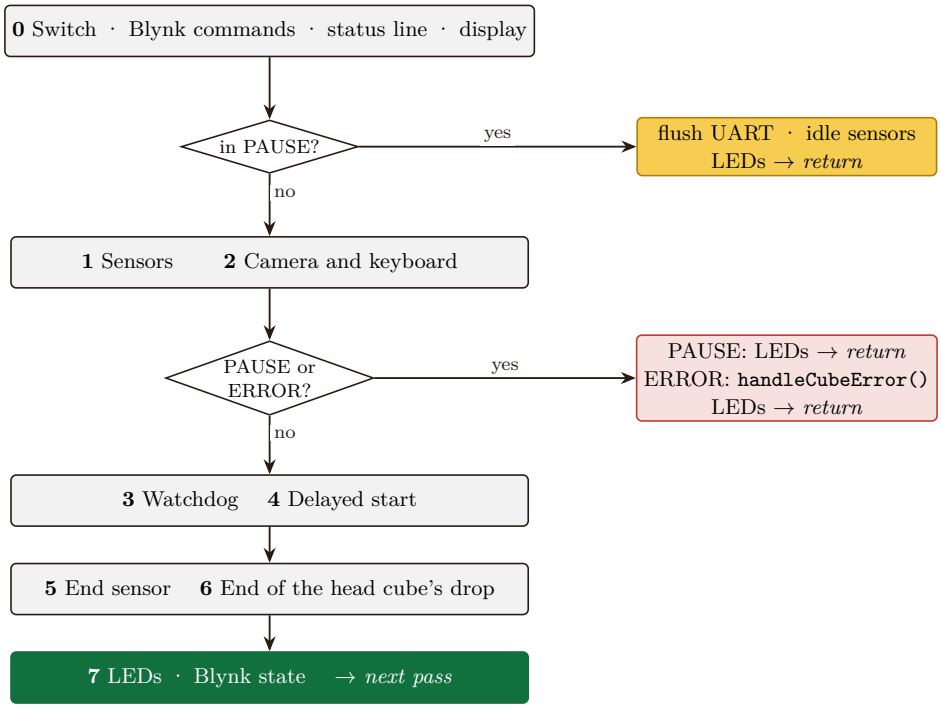



FIGURE 13. Organization of the main loop.

TABLE 15. Pin assignment of the master ESP32.

| Component | Pin | Signal or remark |
|---|---|---|
| TB6600 driver | GPIO5 | PUL (pulses, LEDC PWM channel 0) |
| | GPIO2 | ENA (active level: `LOW`); ESP32 strapping pin |
| | (not driven) | DIR wired directly to the supply |
| Servo | GPIO23 | 50 Hz PWM signal, 14 bits (LEDC channel 2) |
| Start laser sensor | GPIO33 | Analog input (ADC1) |
| End laser sensor | GPIO32 | Analog input (ADC1) |
| Green LED | GPIO27 | Belt running |
| Yellow LED | GPIO26 | Belt stopped, paused or in error |
| Red LED | GPIO14 | Alert and emergency stop |
| Resume switch | GPIO12 | Input with internal pull-up resistor, to ground |
| LCD display (I2C) | GPIO21 / GPIO22 | SDA / SCL, address `0x27` |
| Camera link (UART) | GPIO16 / GPIO17 | RX (from the XIAO TX) / TX (unused) |

## 4.4 Table of Blynk virtual pins

## 4.5 Complete scenario: the passage of a green cube

With the default values ($n = 24$ rev/min, $v = 4$ cm/s, $T_{\text{drop}} = 1250$ ms), table 17 follows a green cube from its detection to its counting. The instants $t_c$ (reception of the color) and $t_f$ (arrival at the end of the belt) depend on the length of the belt and on the camera.

## 4.6 Test plan

The project is validated step by step, from the component to the system (table 18). Each test program isolates one part so that a fault is easily located.

The recommended order is: sensors (01), motor (02), Blynk (03), full program without camera using the keyboard, then full program with the camera. The display and the servo have no dedicated program: they are exercised from the startup of the main program (“System ready” message, servo in the central position) and during the simulation of a sort.

## 4.7 Limits and perspectives

This section lists, plainly, what the program does not do or does not yet guarantee.

— **Minimum spacing.** If two cubes are closer than $D$, the bin may not be oriented in time; the program reports the case in the log but cannot correct it.

— **Manual bin choice.** The choice from the dashboard is refused only in pause; it remains possible while a cube is falling, which can direct it into the wrong container.

— **No servo position feedback.** The program does not check that the bin has reached its position.

— **Watchdog.** It empties the queue, but the cubes still on the belt are then not counted.

— **Flash memory.** Four writes per sorted cube; to be monitored for continuous use.

TABLE 16. Virtual pins of the dashboard and processing rules.

| Pin | Dir. | Widget | Content or processing rule |
|---|---|---|---|
| V0 | ↑ | Last sorted | Name of the last color sorted (YELLOW, GREEN, BLUE, UNKNOWN) |
| V1 | ↑ | Total sorted | Total of sorted cubes, all colors |
| V2 | ↑ | “Run” LED | 255 if the belt is running, otherwise 0 |
| V3 | ↑ | “Pause” LED | 255 if the belt is not running |
| V4 | ↑ | “Alert” LED | 255 in pause or in error |
| V5 to V8 | ↑ | Counters | Yellow (V5), green (V6), blue (V7), unknown (V8) |
| V10 | ↕ | Play / Pause | 0: emergency stop (“DASHBOARD”); 1: resume if paused, otherwise the button is set back to ON |
| V11 | ↑ | Chart | Total sorted over time |
| V12 | ↑ | “Logs” terminal | Timestamped log; input typed in this terminal is ignored |
| V13 | ↕ | “Reset DB” | ON arms the reset; goes back to OFF after each attempt |
| V15 | ↕ | Bin (list) | Follows the real bin; a manual choice is applied if valid and not in pause, otherwise the list is set back to the real bin |
| V16 | ↕ | Confirmation code | Reset code, cleared after each entry |

TABLE 17. Walkthrough of a green cube, from detection to counting.

| Instant | Event | Effects |
|---|---|---|
| $t = 0$ | Edge on the start laser | Cube no. 1 created (color `?`); display “New cube / On belt: 1”; delayed start scheduled |
| $t = 300$ ms | End of the start delay | Motor started at 80 Hz, belt at 4 cm/s, green LED on |
| $t = t_c$ | `V` byte received from the camera | Cube no. 1 becomes green; the watchdog is re-armed |
| $t = t_f$ | Edge on the end laser | Arrival recorded (`dropTime`); head cube with a known color: servo at 180° (pulse 2400 $\mu$s, duty 1965); V15 set to index 2; display “BIN: / GREEN” |
| $t = t_f + 1250$ ms | Drop delay elapsed | Display “ITEM SORTED / GREEN”; green counter + 1 saved in memory; Blynk: V0 = GREEN, V1, V6, V11 and log line “GREEN sorted \| total: n” |
| Right after | Queue empty | Motor stopped; yellow LED; system waiting for a new cube |

TABLE 18. Test programs and debugging facilities.

| Program | Purpose of the test |
|---|---|
| `01_laser_ldr_detection` | Reading of the analog value of the laser sensor and display on the serial monitor; used to find the high and low levels and to choose the threshold $S$ |
| `02_stepper_tb6600` | Control of the motor through the TB6600 driver; checks the PUL, DIR, ENA wiring and the speed |
| `03_blynk_communication` | Exchanges with the dashboard, in both directions, with the same virtual pins as the main program; serial commands `J V B X` (simulated sort), `R` (reset), `S` (status) |
| `ESP32_master` (keyboard simulation) | The whole sorting process without a camera: type `J`, `V`, `B`, `X` or `A` in the serial monitor |
| `ESP32_master` (raw UART, status line) | Each byte received from the camera is logged; the full state is displayed every 5 s |

— **Test options active by default.** Keyboard simulation and raw UART logging are enabled; they should be disabled in operation.

— **Single direction of rotation.** The DIR input of the driver is wired to the supply.

The perspectives retained by the club are: a better camera with controlled lighting and shape sorting with OpenCV, more colors, a faster belt and several bins, and a hosted web dashboard with a cloud database (for example Firebase) to keep a long history, beyond what Blynk allows.

# PART 5: CONCLUSION

The study has described the firmware of the master ESP32 and the logic on which it relies. The approach is the same for each module: identify a precise problem, choose the suitable technique, then check by calculation that it fits the values of the project. Five ideas sum up the whole:

— **Compare instants rather than wait**: all delays (start, debounce, blinking, drop) are differences of `millis()`, which keeps the loop always listening;

— **React to edges, not levels**: combined with debounce, they turn a noisy analog signal into reliable events, provided that $\Delta t_{\mathrm{db}} < D/v$;

— **Track the cubes in a FIFO queue**: each cube keeps its color and its state, and the bin follows the order of the cubes without ever being moved during a drop;

— **Isolate the network**: the Blynk task, on its own core, communicates with the sorting loop through a message queue and flags, so that a failing Wi-Fi cannot stop the belt;

— **Put safety first**: the emergency stop is tested before anything else, the timers are frozen during the pause, and an unknown color causes a safe stop rather than a wrong sort.

This study also highlights acknowledged limits (minimum spacing of the cubes, unlocked manual bin choice) listed in part 4. The perspectives are improving the vision, increasing the capacity (colors, speed, bins) and a production history hosted beyond Blynk.

# BIBLIOGRAPHY

[1] Espressif Systems, *Arduino-ESP32: documentation* (LEDC, UART, I2C, ADC, Preferences, WiFi). `https://docs.espressif.com/projects/arduino-esp32/`

[2] Espressif Systems, *ESP-IDF Programming Guide*: FreeRTOS, tasks and message queues. `https://docs.espressif.com/projects/esp-idf/`

[3] FreeRTOS, *FreeRTOS Reference Manual*: tasks, queues. `https://www.freertos.org/`

[4] Blynk, *Blynk Documentation*: templates, virtual pins, `BLYNK_WRITE`. `https://docs.blynk.io/`

[5] Seeed Studio, *Getting Started with Seeed Studio XIAO ESP32S3 Series*, Seeed Studio Wiki. `https://wiki.seeedstudio.com/xiao_esp32s3_getting_started`

[6] Maker Group Global LLC, *TB6600 Stepper Motor Driver*, V1.0, July 2018.

[7] Manufacturer not specified, *SG90: 9 g Micro Servo*, datasheet.

[8] Hitachi Semiconductor, *HD44780: Dot Matrix Liquid Crystal Display Controller/Driver*, datasheet.

[9] NXP Semiconductors, *UM10204: I2C-bus specification and user manual*.

[10] Club d'Informatique Industrielle IGE × EPAC, *Electrical system study: automatic color sorting system*, 12 October 2026.

[11] Club d'Informatique Industrielle IGE × EPAC, *Tri Industriel: automatic color sorting of cubes, remotely supervised in real time*, presentation (15 slides).

[12] Project source code: `ESP32_master.ino`, `01_laser_ldr_detection.ino`, `02_stepper_tb6600.ino`, `03_blynk_communication.ino`.

### Illustrations

The diagrams of this study were produced by the author; the communication diagram reproduces the one in the project presentation.

# APPENDIX A: FUNCTION INDEX

TABLE 19. Functions of the program, module they belong to and role.

| Function | Module | Role |
|---|---|---|
| `elapsedSince` | — | Time elapsed since a stored instant |
| `cubePresent` | 2 | Cube in front of the sensor, based on the last value |
| `risingEdge` | 2 | Rising edge with debounce |
| `initSensors` | 2 | Initializes the "previous" state with the real state |
| `addNewCube` | 3 | Adds a cube to the queue, schedules the start |
| `removeCubeAt` | 3 | Removes a cube at a given position |
| `resetQueue` | 3 | Empties the queue |
| `handleEndOfBelt` | 3 | Handles the arrival of a cube at the end sensor |
| `advanceToNextCube` | 3 | Prepares the next cube after a sort |
| `handleCameraUart` | 4 | Reads all the available bytes |
| `readCameraByte` | 4 | Reads a byte and logs it |
| `receiveByte` | 4 | Routes a byte: stop or color |
| `receiveColor` | 4 | Assigns the color to the oldest cube without a color |
| `stepFrequencyHz` | 5 | Pulse frequency for the current speed |
| `computeDropDelayMs` | 5 | Drop delay in milliseconds |
| `startMotor`, `stopMotor` | 5 | Starts or cuts the pulse train |
| `setRpm` | 5 | Changes the speed |
| `servoAngle` | 6 | Positions the servo at an angle |
| `angleFor`, `menuIndex` | 6 | Color to angle, color to list index |
| `orientBin` | 6 | Orients the bin for a color |
| `colorName`, `isUnknown` | 6 | Name of a color, non-sortable color |
| `enterPause`, `exitPause` | 7 | Emergency stop and resume with timer freezing |
| `handleToggleSwitch` | 7 | Reading and debounce of the switch |
| `triggerCubeError` | 7 | Stop on an unknown cube |
| `handleCubeError` | 7 | Wait for removal, automatic resume |
| `signalActivity` | 7 | Re-arms the watchdog |
| `updateLeds`, `updateAlert` | 8 | LED control and blinking |
| `triggerAlertX` | 8 | Starts the three blinks of an `X` |
| `writeLcd`, `showLcd`, `refreshLcd` | 8 | Display and periodic rewrite of the screen |
| `printHeartbeat` | 8 | Periodic status line (debugging) |
| `blynkTask` | 9 | Network task of core 0 |
| `pushInt`, `pushTxt`, `termLog` | 9 | Dropping of messages in the queue |
| `timestamp` | 9 | Network time or time since startup |
| `handleBlynkCommands` | 9 | Handles the dashboard commands |
| `sendFullState`, `syncBlynkState` | 9 | Full resynchronization, sending on change |
| `totalSorted`, `saveCounters`, `pushCounters` | 10 | Total, saving to flash, sending to the dashboard |

| Function | Module | Role |
|---|---|---|
| `recordSort`, `resetCounters` | 10 | Counting a sorted cube, reset |
| `setup`, `loop` | 4.2 | Initialization and main loop |